

Team name: The Pantheon Corporation



Table of Contents:

1.0. Team Information and technology prototypes	3
1.1. Team Membership	3
1.2. Team Schedule	4
1.3. Technology Stack Justification	5
2.0. User Stories	6
2.1. extension:	6
2.2. UI/UX:	7
2.3. Gameplay:	9
3.0. Domain Model	14
3.1. Design	14
3.2. Justification	14
1. Game	15
2. Tile	16
3. Player	16
4. Worker	17
5. GodPower	18
6. Building	19
7. Dome	20
8. Level	20
3.3. Assumptions & Other Notable Choices	21
4.0. Prototype	22
4.1. Step 1 and 2 covers how our santorini game is set up	22
4.2. Step 3, 4, and 5 covers how player one completes a single turn (select, move, and build)	23
4.3. Step 6, 7, and 8 covers how player two completes a single turn (select, move, and build)	25
4.4. Step 9, 10, and 11 covers how player one can build on top of another building (second level)	27
4.5. Step 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, and 25 shows how our chosen extension (natural disaster) would play out during a game	28
4.6. Step 26, 27, and 28 briefly shows how a player could win or lose in a game when the winner reaches the third building before the losing player	35
4.7. Step 29, 30, and 31 briefly shows how a player moves down from a building	37

1.0. Team Information and technology prototypes

1.1. Team Membership

	Member 1	Member 2	Member 3
Name	Aldrin Akeyla Riadiputra (Lead Sprint 1, 2, 3)	Hagen Yovando	Reynard Tjakrawadi
Contact Detail	aria0009@student.m onash.edu	hyov0001@student.mona sh.edu	rtja0004@student.m onash.edu
Technical and Professional Strengths	- Leadership skills - Discipline - Experience in Java programming language	- Communication - Problem-solving - Self-learning - Experience in Java	- Teamwork - Flexible - Experience in OOP using Java
Fun Facts	- I was in a school band	- I spilled my cereal on my laptop	- I play games

1.2. Team Schedule

Meeting Schedule:

	Date, Time, Location	Members Present	Meeting agenda	Workload Distribution
Meeting 1	Date: 18th March 2025 Time: 8:00 PM - 9:00 PM Location: Turner Hall	Aldrin Hagen Reynard	Complete Team information and technology prototypes	Workload distributed evenly to all members
Meeting 2	Date: 21st March 2025 Time: 5:00 PM - 9:00 PM Location: Turner Hall	Aldrin Hagen Reynard	Complete User story and INVEST criteria	Workload distributed evenly to all members
Meeting 3	Date: 25 March 2025 Time: 6:00 PM - 9:00 PM Location: Campbell hall	Aldrin Hagen Reynard	Start Domain Model	Workload distributed evenly to all members
Meeting 4	Date: 30 March 2025 Time: 2:00 PM - 6:00 PM Location: Online via Discord	Aldrin Hagen Reynard	Finish the Domain Model, write justification and re-adjust investment criteria	All: Finish the Domain model Aldrin: Re-adjust investment criteria Hagen and Rey: Write a domain model justification.
Meeting 5	Date: 4 April 2025 Time: 2:00 PM - 6:00 PM Location: Online via Discord	Aldrin Hagen Reynard	Finish prototype And a Jar file	The workload distributed evenly to all members

1.3. Technology Stack Justification

Programming language used: Java

Our team has chosen Java for the project because it aligns closely with our current expertise. The majority of our team has experience working with Java in the previous courses therefore we are familiar with this programming language. This familiarity will allow us to minimise any technical and learning issues when doing our project. However, we anticipate one of our members, Hagen, might need a little more support since he has limited skills in collaborative coding with git before.

2.0. User Stories

2.1. extension:

1. As an event, I want a natural disaster to occur randomly, so that the game board changes unpredictably, adding new strategic depth.

How user story meets INVEST criteria:

- It is **independent** since it functions as an isolated feature.
- It is **Negotiable** since the disaster can be adjusted.
- It is **valuable** since it adds strategic depth to the game.
- It is **estimable** since the boundaries of the feature are defined.
- It is **small** since it focuses on introducing a single thing only.
- It is **testable** since the functionality can be verified by many testing scenarios.

2. As a player, I want a new god power that allows me to manipulate the game state, so that I can disrupt my opponent's plans or gain an advantage.

How user story meets INVEST criteria:

- It is **independent** since the god power can be developed as an isolated feature
- It is **negotiable** since the mechanics of the power are open to refinement
- It is **valuable** since it provides players with strategic tools that can shift game dynamics.
- It is **estimable** since the boundaries of the feature are well defined
- It is **small** since it focused on introducing well defined ability
- It is **testable** since

3. As a board, I want an option to have different configurations, so that players can enjoy multiple board configurations.

- It is **negotiable** since the number and types of configuration can be changed
- It is **valuable** it adds variety and replayability
- It is **estimable** since implementation of different board shapes or sizes has a well define scope
- It is **testable** since we can verify if everything behaves correctly

2.2. UI/UX:

1. As a player, I want a clean and visually appealing interface so that I can easily distinguish game elements.

How user story meets INVEST criteria:

- It is **independent** since it is separate with the game logic
- It is **negotiable** since it is quite subjective so it can be refined via feedback
- It is **valuable** by enhancing the overall player experience.
- It is **estimable** since we can estimate based in the number of screen and elements
- It is **small** since it is only UI visual improvements
- It is **Testable** since the we can quality test it to confirm the clarity and aesthetics

2. As a player, I want clear visual indicators for my selected worker so that I know which piece I am moving.

How user story meets INVEST criteria:

- It is **independent** because it doesn't depend on game rules and only on UI interaction
- It is **negotiable** since the style can be adjusted
- It is **valuable** since it prevents mistakes during movement and improves usability for the users
- It is **estimable** since the implementation effort can be estimated
- It is **small** since it is just simple UI
- It is **testable** since we can easily check if UI works or not

3. As a player, I want to be able to view my god's power description during the game, so that I can remember how to use it effectively.

How user story meets INVEST criteria:

- It is **independent** since it is only tied to the UI and not game logic
- It is **negotiable** since the presentation style can be adjusted
- It is **valuable** since it supports the strategic decision-making
- It is **estimable** since it is just a small UI element and it is straightforward to estimate.
- It is **small** since it's just a single feature.
- It is **testable** since we can easily check whether the UI is showing or not

4. As a worker, I want a different worker model for each player, so that I can differentiate my worker with my opponent's.

How user story meets INVEST criteria:

- It is **independent** since it only affects design and not other game logic
- It is **negotiable** since the design can vary based on preferences
- It is **valuable** since it reduces confusion
- It is **estimable** since the visual change is measurable
- It is **small** since there are only two workers for each player
- It is **testable** since we can confirm if each player sees their unique worker model

5. As a building, I want to have a different structure style for each level of the building so that I can differentiate between the levels.

How user story meets INVEST criteria:

- It is **independent** since it only affects the visuals
- It is **negotiable** since visual style can be iterated
- It is **valuable** for the player to clearly differentiate the towers to plan their next move
- It is **estimable** since it is easy to estimate visual layering
- It is **testable** since we can manually check visual clarity of each structure.

2.3. Gameplay:

1. As a player, I want to be able to play the game without bugs so that I am not distracted while playing the game

How user story meets INVEST criteria:

- It is **independent** since it's a quality of life improvement affecting all gameplay.
- It is **negotiable** since bug fixes can be prioritized and adjusted.
- It is **valuable** because a bug-free experience ensures smooth gameplay.
- It is **estimable** since we can estimate the complexity based on the number of bugs.
- It is **small** since it targets stability and does not involve adding new features.
- It is **testable** since we can verify that bugs are resolved through playtesting.

2. As a player, I want to place my workers on the board before the game starts, so that I can determine my initial strategy.

How user story meets INVEST criteria:

- It is **independent** since it doesn't depend on other mechanics.
- It is **negotiable** since placement rules and limits can be adjusted.
- It is **valuable** since it introduces a strategic element early in the game.
- It is **estimable** since the feature is well-defined and easy to implement.
- It is **small** since it focuses only on the worker placement phase.
- It is **testable** since we can verify correct worker placement functionality.

3. As a worker, I want to be able to move so that I can position myself advantageously on the board.

How user story meets INVEST criteria:

- It is **independent** since movement is a core feature that doesn't rely on other mechanics.
- It is **negotiable** since movement rules can be refined.
- It is **valuable** since positioning is key to gameplay strategy.
- It is **estimable** since movement logic is straightforward to implement.
- It is **small** since it's a focused feature on just worker movement.
- It is **testable** since we can check that the movement works as expected.

4. As a worker, I want to be able to place a building so that I can advance the game and create strategic opportunities.

How user story meets INVEST criteria:

- It is **independent** since it works as its core game action
- It is **valuable** since it enables fundamental gameplay
- It is **estimable** since it is a simple logic
- It is **small** since it is a single action
- It is **testable** since we can confirm the buildings are placed only on valid tiles

5. As a game, I want to have a win and lose condition so that there is a fair and competitive challenge.

How user story meets INVEST criteria:

- It is **independent** since Logic can be triggered based on board state, not tied to UI.
- It is **negotiable** since The win condition messaging or animations can vary.
- It is **valuable** since Core goal of the game is critical to complete the experience.

- It is **Estimable** since Logic is finite and well-defined (reach level 3 or no moves).
- It is **small**. It can be implemented as a trigger condition.
- It is **testable** and Easily verifiable when winning/losing conditions are met.

6. As a god power, I want to provide players with abilities so that they can influence the board dynamically.

How user story meets INVEST criteria:

- It is **independent** since god powers are modular and triggered separately from standard rules.
- It is **negotiable** since power effects and activation timing can be adjusted.
- It is **valuable** since it adds uniqueness and variety to gameplay.
- It is **estimable** since powers have specific, defined effects.
- It is **small** since each power can be implemented individually.
- It is **testable** since you can confirm whether each god's power functions as expected.

7. As a player/worker/board/game, I want turns to be clearly defined so that gameplay is structured and fair.

How user story meets INVEST criteria:

- It is **independent** since it operates separately from specific player actions.
- It is **negotiable** since turn indicators and UI elements can be changed.
- It is **valuable** since it ensures a fair gameplay flow.
- It is **estimable** since the turn logic is straightforward.
- It is **small** since it manages only which player is active.
- It is **testable** since only one player can act at a time.

8. As a player, I want to be notified when I attempt an illegal move so that I understand why my action is not allowed.

How user story meets INVEST criteria:

- It is **independent** since it's triggered by a rule violation, not tied to the success path.
- It is **negotiable** since the notification style can vary (popup, tooltip, text).
- It is **valuable** since it improves user feedback and understanding.
- It is **estimable** since it's easy to identify invalid move conditions.
- It is **small** since it only involves a single response per invalid move.
- It is **testable** since errors can be intentionally triggered and verified.

9. As a player, I want to be able to choose god power so that each game offers variety while still giving me some strategic choice.

How user story meets INVEST criteria:

- It is **independent** since it occurs during setup and doesn't affect turn logic.
- It is **negotiable** since the number of choices and randomness method can be changed.
- It is **valuable** since it enhances replayability and balance.
- It is **estimable** since the draw logic and choice system are clearly defined.
- It is **small** since it only impacts game setup.
- It is **testable** since selection flow and randomness can be verified.

10. As a player, I want to be assigned a different model of workers so each player worker can be differentiated.

How user story meets INVEST criteria:

- It is **independent** since it involves only cosmetic aspects.
- It is **negotiable** since model styles or colors can be swapped.
- It is **valuable** since it avoids confusion between players' pieces.
- It is **estimable** since the models are static assets with no logic.
- It is **small** since it affects only visuals.
- It is **testable** since selected models can be visually confirmed.

11. As a worker, I want to be able to place a dome on the third floor, so that I can block the opponents moves.

How user story meets INVEST criteria:

- It is **independent** since dome placement is a standalone rule after building.
- It is **negotiable** since visual or sound cues can vary.
- It is **valuable** since domes are part of strategic defense.
- It is **estimable** since the logic is triggered only at level 3.
- It is **small** since it's a single rule with a simple condition.
- It is **testable** since dome placement can be checked on level 3 buildings.

12. As a dome, I want to be able to block workers from going up to the third level, so that I can't prevent the player from winning the game.

How user story meets INVEST criteria:

- It is **independent** since preventing access to the third level doesn't change other building or moving logic.
- It is **negotiable** since how the dome interacts with levels or timing can be adjusted for balance.
- It is **valuable** since it directly prevents a straightforward win, adding tension and strategy.
- It is **estimable** since blocking a tile is a simple rule check.
It is **small** since it specifically focuses on preventing movement onto the third level.
- It is **testable** since you can verify whether workers are indeed blocked when a dome is in place.

13. As a player, I want to be able to confirm or undo my moves before moving so that I can rethink my moves again.

As a player, I want to be able to confirm or undo my moves before moving so that I can rethink my moves again.

How user story meets INVEST criteria:

- It is **independent** since it only affects the player's input phase, not core rules.
- It is **negotiable** since how many undos are allowed or the timing can be adjusted.
- It is **valuable** since players can fix mistakes or reconsider strategies.
- It is **estimable** since adding an undo confirmation is a well-understood UI pattern.
- It is **small** since it just involves tracking and reverting a single move.
- It is **testable** since you can simulate an attempted move, undo it, and confirm the game state has rolled back.

How user story meets INVEST criteria:

- It is **independent** since it applies to the UI before the move execution.
- It is **negotiable** since the number of undos or timing can be configured.
- It is **valuable** since it reduces errors and improves user control.
- It is **estimable** since rollback and confirmation are well-known patterns.
- It is **small** since it applies only within the same turn.
- It is **testable** since you can attempt to undo or confirm before committing.

14. As a player, I want to be able to make sure that the opponent can't move 2 or 3 levels up in 1 move unless they have a god power so that the the game can be more challenging

How user story meets INVEST criteria:

- It is **independent** since it's a movement rule, not tied to a specific feature.
- It is **negotiable** since exceptions like god powers can be added.
- It is **valuable** since it maintains game balance and fairness.
- It is **estimable** since movement rules are easy to scope.
- It is **small** since it is just a condition in the move validation.
- It is **testable** since illegal multi-level jumps can be blocked and checked.

15. As a player, I want to be able to resign so that I don't waste my time playing a losing game

How user story meets INVEST criteria:

- It is **independent** since it triggers a game state change separate from gameplay.
- It is **negotiable** since UI and confirmation flow can be customized.
- It is **valuable** since it respects player time and gives control.
- It is **estimable** since it involves a single game-ending button.
- It is **small** since it only sets the game to end.
- It is **testable** since resigning should always end the game correctly.

16. As a player, I want to be able to skip my god's power in a bad position so that I have a better chance of winning.

How user story meets INVEST criteria:

- It is **independent** since skipping power use is separate from core actions.
- It is **negotiable** since whether skipping is optional or limited can change.
- It is **valuable** since forced power use could be disadvantageous.
- It is **estimable** since skipping is a simple conditional choice.
- It is **small** since it's just a toggle or decision point.
- It is **testable** since the player can choose whether to activate the power.

17. As a player with the god power Artemis, I want to be able to move my workers 1 additional time but not moving back to my original space so that I can use my god power effectively to increase my chances of winning.

How user story meets INVEST criteria:

- It is **independent** since the extra move can be added without modifying other god powers.
- It is **negotiable** since restrictions (like not moving back to the same tile) can be adjusted for balance.

- It is **valuable** since it offers a strategic advantage, enhancing gameplay variety.
- It is **estimable** since granting an extra move involves straightforward logic.
- It is **small** since it's just an additional action within the normal turn.
- It is **testable** since you can verify that two moves are made without returning to the original tile.

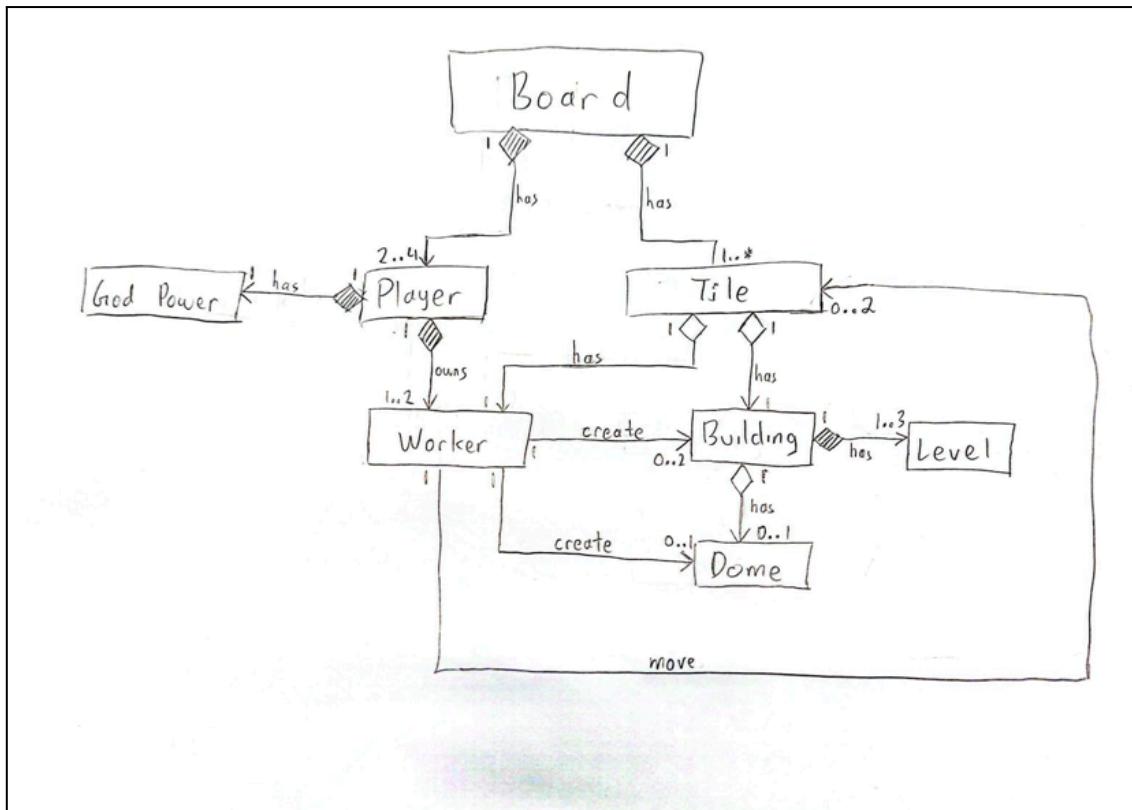
18. As a player with the god power Atlas, I want to be able to place a dome at any given level so that my opponent can't move to the space that I have built a dome at and increase my chances to win.

How user story meets INVEST criteria:

- It is **independent** since placing domes doesn't require altering the standard building rules.
- It is **negotiable** since you could limit how many domes can be placed per turn or game.
- It is **valuable** since blocking an opponent's movement can shift the game's momentum.
- It is **estimable** since placing a dome at any level is a simple extension of normal building.
- It is **small** since it only adds a single, specialized action to the gameplay.
- It is **testable** since you can confirm domes are placed correctly and opponents can't enter those spaces.

3.0. Domain Model

3.1. Design



3.2. Justification

- Were there any specific choices that you had to make while modelling the domain and WHY?

-

In the diagram, we made a parent class called action that contains all the actions that the worker can make since the workers can make multiple actions. This can also make our game more extendable since if we want to add a new type of action, we can add it under action.

Same goes for the parent class building, since there are multiple types of building blocks for the building and it can be generalized as a building class and if we want to add another type of building an example is a whirlpool from god powers, it can have the basic attributes of a building.

The class GameObject is a parent class to building and worker. These 2 are a child of GameObject since both these items exist in the board and both share the same attribute for an example, its location in the board.

- Explain any assumptions you have made, as well as any other part of your domain model that you feel warrants a justification as to WHY you have modelled it that way.
 - **Assumptions**
 Since in the class diagram we make workers, god powers and building as a child of gameObject, we assume that gameObject is something that can make changes to the board. We also assume that the extra moves that the workers get from god powers to be a move created by the god powers itself.

1. Game

- **Justification:**
 - The *Game* entity acts as the overarching container for the match. It manages and coordinates all other elements such as the *Tiles* , *Players*, and the global rules or statuses (like current turn, win/loss conditions, etc.).
 - Having a single *Game* entity clarifies the scope—when you start a Santorini match, you create a *Game*.
- **Modeling Choice:**
 - We treat *Game* as a “root” entity. Because Santorini is turn-based, you need a central way to keep track of start/end states, the turn order, and references to key sub-elements.
- **Relationship:**
 - “Game” has a composition relationship with both tile and player since the Santorini game itself needs a board which can be made using multiple tiles and the game cannot start without any players.

2. Tile

- **Justification:**

- Santorini is played on a 5×5 board (or grid), but conceptually it represents the environment or “arena” in which all gameplay happens.
- The board of santorini can be represented with tiles, by doing this, we can modify the size of the board which can be an extension for the game. The tile can contain more than 1 building but can only contain 1 worker.

- **Relationships:**

- “The Tile “has” a Worker” and “The Tile “has” a Building“. “Has” means that the Tile is in an Aggregation relationship with both the *worker* and the *building*. In the tile, there may exist a Building, a Worker or both of them and it can also exist without having anything in it.

- **Modeling Choice:**

- We choose to have a Tile as a single entity because by doing this, the board can be expanded infinitely by just adding more of the Tile entities.
- We choose to store the workers and building in each Tile so that the Tiles can communicate with each other when moving a worker or building a level.

3. Player

- **Justification:**

- In a 1v1 Santorini setup, there are exactly two players (though the board game can support more players in some expansions, we focus on the simplest 1v1).
- Each player is responsible for controlling 2 workers.
- Players take turns performing actions.
- Each player has a god power.

- **Relationships:**

- “*Each Player* has a *God Power*” this means that the player always has a God Power and the Player cannot exist without a God Power since the player must choose a God Power before starting the game.
- “*Each Player* owns exactly two *Workers*” this means that the player always has 2 workers and the Player cannot exist with only 1 since this is a 2 player game and the player must place 2 workers before starting the game.

- **Modeling Choice:**

- We limit the cardinality to exactly 2 *Workers* per *Player*, in line with standard Santorini rules.
- We keep *Player* separate from *Worker* so that we can track each user’s name, any special abilities, or win condition checks at the *Player* level, while *Worker* is purely a piece on the board.

4. Worker

- **Justification:**

- Each player’s physical presence on the board is represented by two workers. They must be placed on the board, can move, build, etc.

- **Relationships:**

- “Each *Worker* is owned by exactly one *Player*.”
- “Each *Worker* moves using 0 to 2 *Tile*” this means that the worker has an associative relationship with the *Tile* entity and can move up to 2 times using the *Tile* entity or not move at all.
- “Each *Worker* creates 0 to 2 *buildings*” this means that the worker has an associative relationship with the *Building* entity and can build either no building at all or up to 2 buildings.
- “Each *Worker* creates 0 to 1 *domes*” this means that the worker has an associative relationship with the *Dome* entity and can build either a *Dome* or no *Dome* at all.

- **Modeling Choice:**

- We choose to use an associative relationship between the worker and the *Tile* so that when the worker moves, it does not need an additional entity that enables

movement. Using this method reduces complexity and removes redundancy.

- We choose to use an associative relationship between the worker and the dome to limit the number of domes that the worker can build.
- We choose to use an associative relationship between the worker and the building so that the worker can build both the dome and the level, we limit the number of buildings that can be built according to the available God Powers.

5. GodPower

- **Justification:**

- In Santorini, each player can be assigned a god power at the start of the game. This power modifies how you can move, build, or otherwise manipulate the board.

- **Relationships:**

- “Each *God Power* is owned by exactly one *Player*”

- **Modeling Choice:**

- By modeling *GodPower* as a subclass of *GameObject*, we unify all board-affecting objects under one hierarchy. This means the board logic can treat god powers as “something that can change the game state.”
- We choose to make the God Power entity to only be related to the Player entity to reduce complexity, we want to make sure that the entity that is responsible for the Worker entities is only the Player entity.

6. Building

- **Justification:**

- A central part of Santorini is placing building levels (up to three blocks plus a dome).
- *Building* is a physical structure on the board, it could be a level or a dome.

- **Relationships:**

- “A *Building* is owned by a *Tile*” this means that the building can exist in a *Tile* or not exist at all.
- “A *Building* is created by a *Worker*” this means that the building can only exist if the *Worker* entity creates it.
- “A *Building* has a *Dome*” this means that the *Building* entity has an aggregation relationship with the *dome* since a building can still exist without a dome.
- “A *Building* has a *Level*” this means that the building entity has a composition relationship with the *Level* since a building cannot exist without having a level.

- **Modeling Choice:**

- We keep *Building* as a separate class because it has unique rules for how it can be placed or stacked, and it physically occupies a board space.
- .We use a composition or aggregation relationship to increase the usability of this entity.

7. Dome

- **Justification:**

- In the Santorini game, a dome is something that the workers can't stand on and since it is a structure, it can be a part of a building.

- **Relationships:**

- "*The Dome* is owned by a *building*" this means that the Dome can only exist if the building exists.

- **Modeling Choice:**

- Since the relationship between dome and building is an aggregation, we can implement reusability from the building entity

8. Level

- **Justification:**

- Levels is a main part of a building in the Santorini Game since a building cannot exist without having a level.

- **Relationships:**

- "*Level* is owned by a *Building*" this means that the Building entity can exist if there is a Level entity.

- **Modeling Choice:**

- Since the relationship between dome and building is a composition, we can implement reusability from the building entity

3.3. Assumptions & Other Notable Choices

1. Each player has exactly two workers (standard Santorini rules).
2. A tile can only hold one worker at a time (no stacking).
3. Buildings can only be placed on empty tiles, and they must be constructed sequentially (level 1 → level 2 → level 3 → dome).
4. God Powers are immutable during a game—a player gets one power and cannot change it mid-game
5. Moves must be validated against adjacency and height rules (workers cannot move up more than one level per turn).

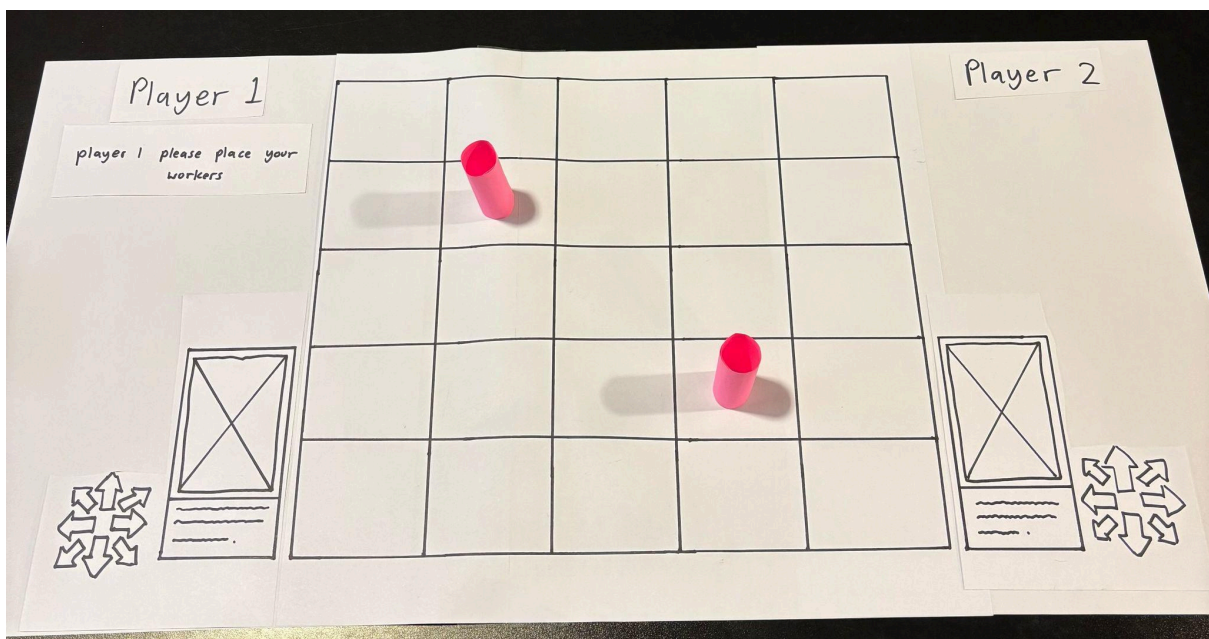
4.0. Prototype

This prototype should cover all the key interaction scenarios of the Santorini game:

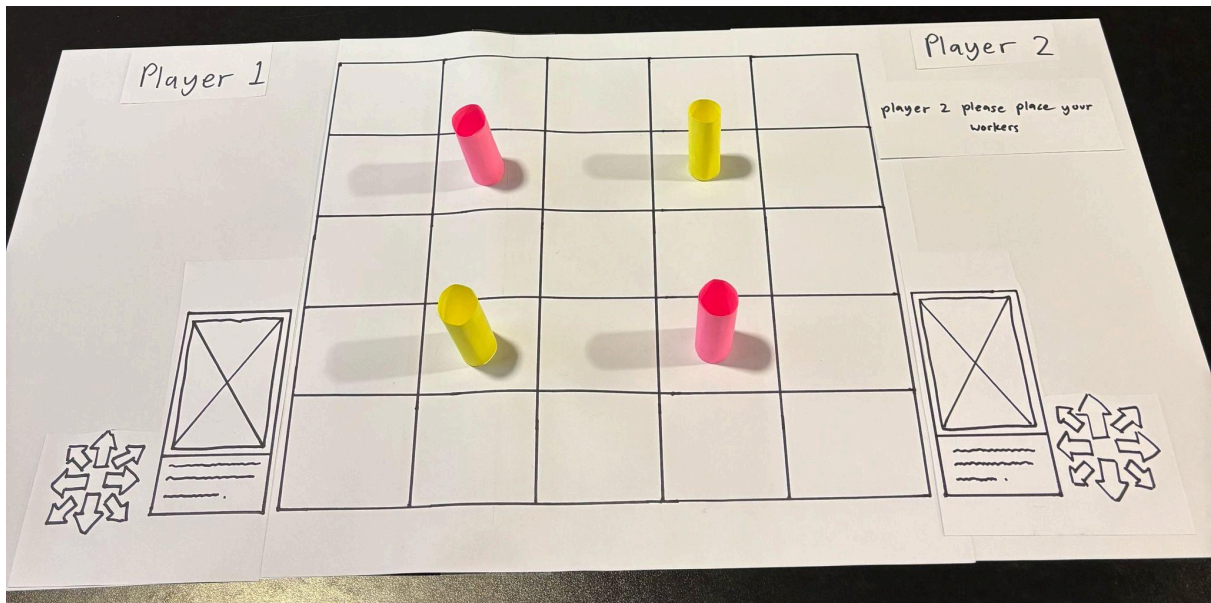
- Setting up the game with starting workers
- How a complete turn for player 1 and 2 will run
- How players build
- How players build on top of a building
- How our extension will play out
- Winning and losing condition
- How players can move down from a building

4.1. Step 1 and 2 covers how our santorini game is set up

1.

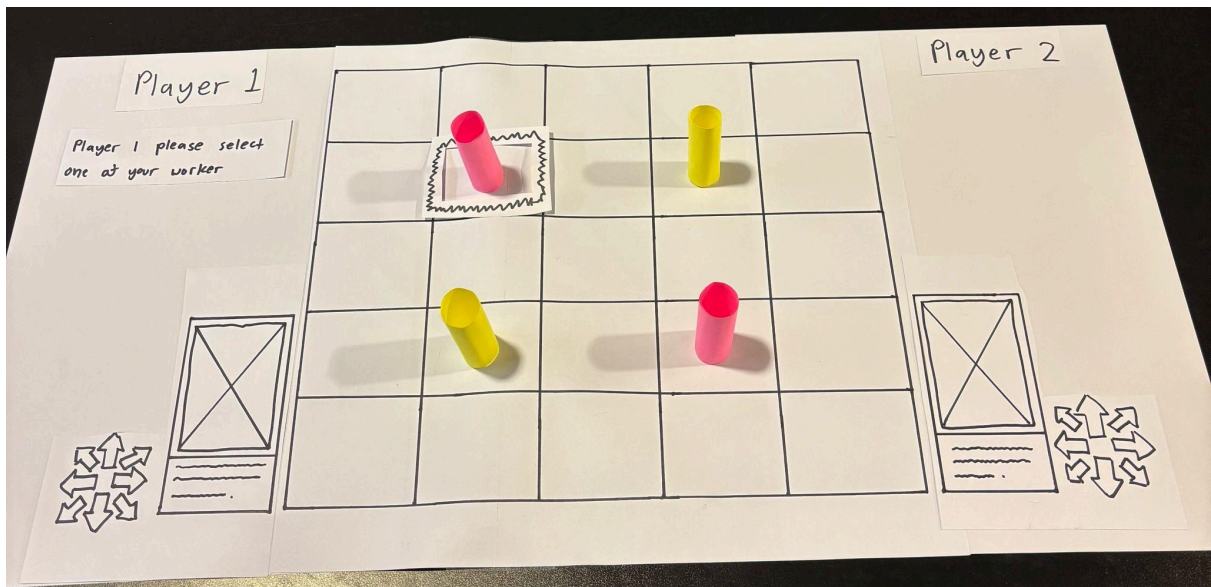


2.



4.2. Step 3, 4, and 5 covers how player one completes a single turn (select, move, and build)

3.



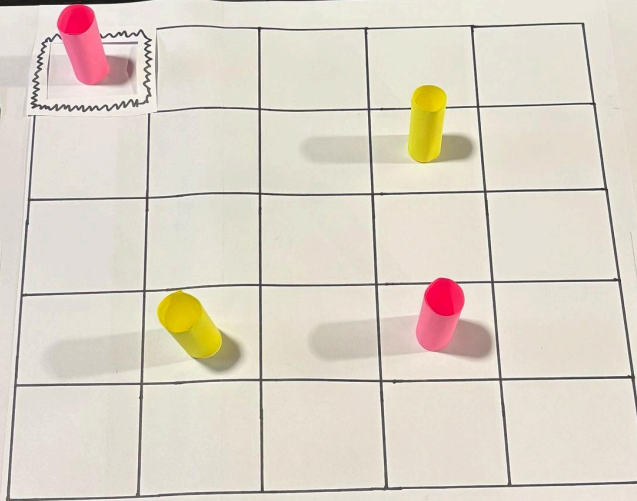
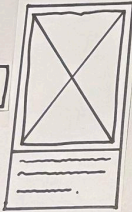
4.

Player 1

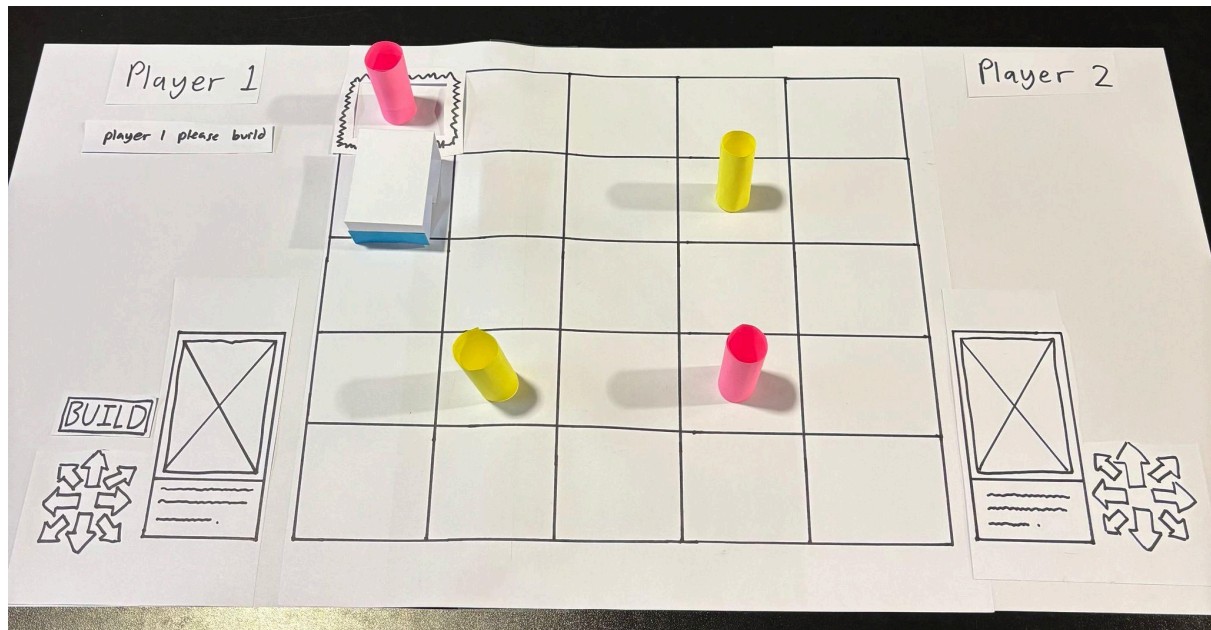
Player 1 please move your
worker

Player 2

MOVE

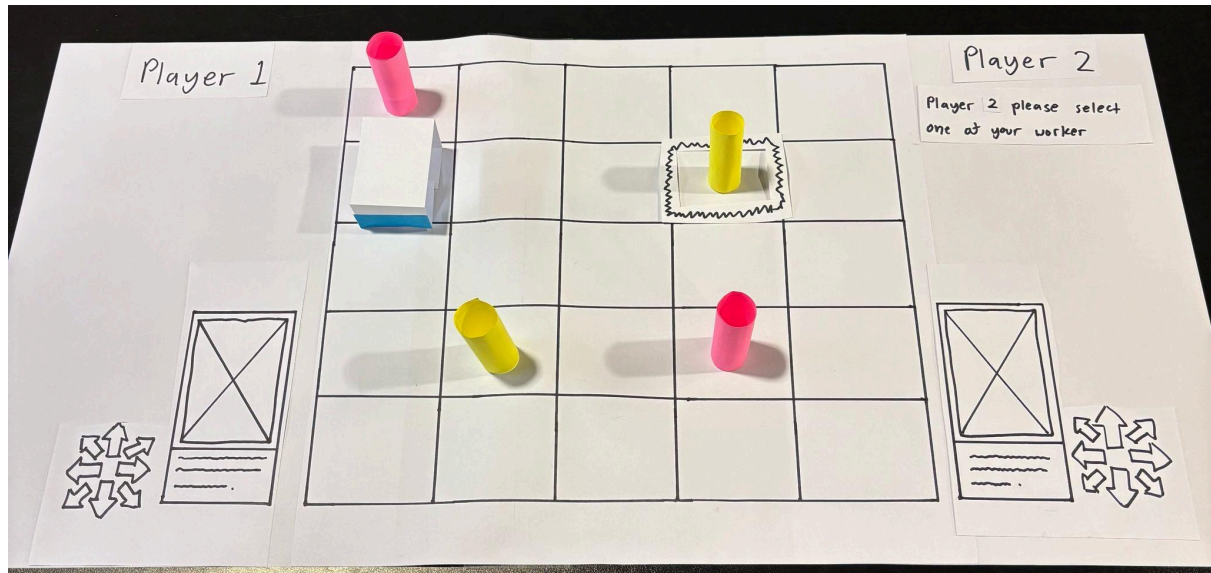


5.

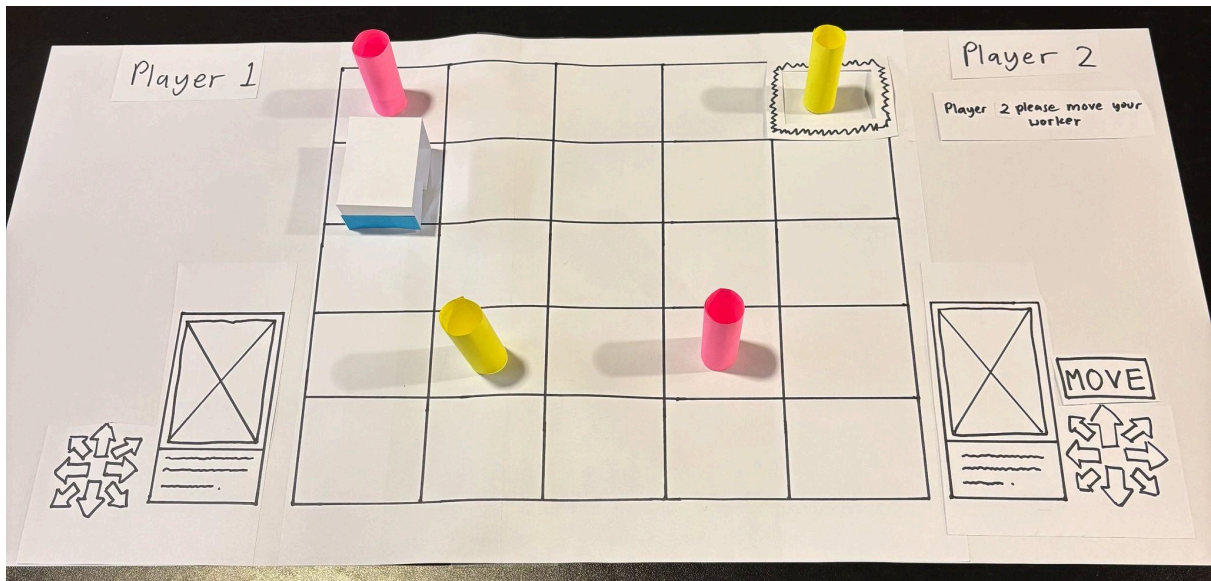


4.3. Step 6, 7, and 8 covers how player two completes a single turn (select, move, and build)

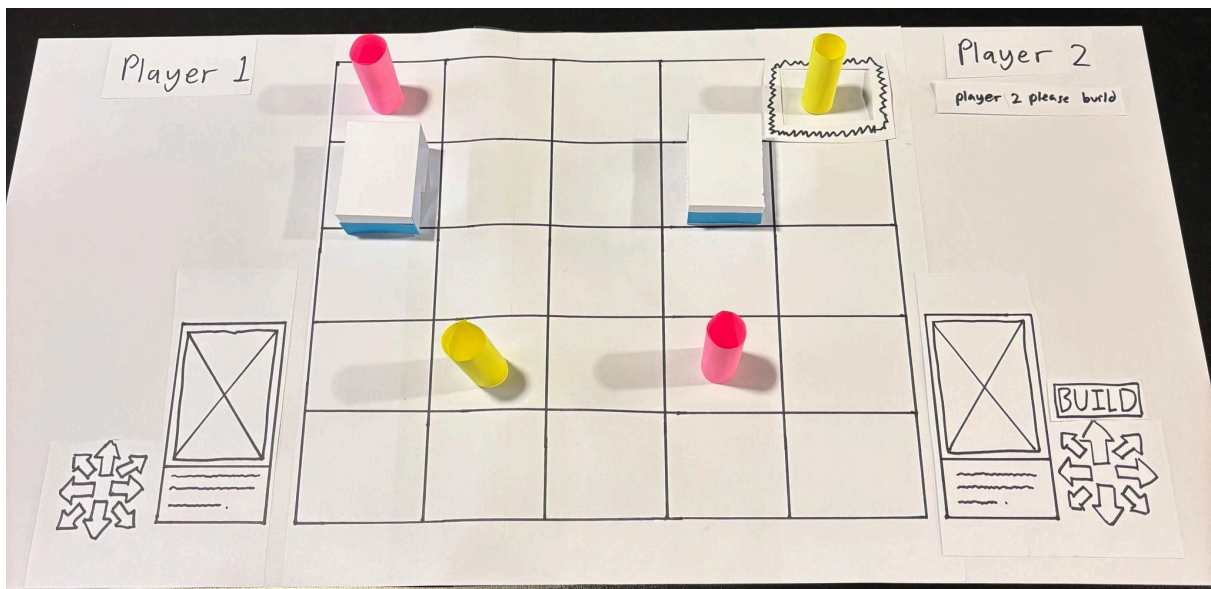
6.



7.

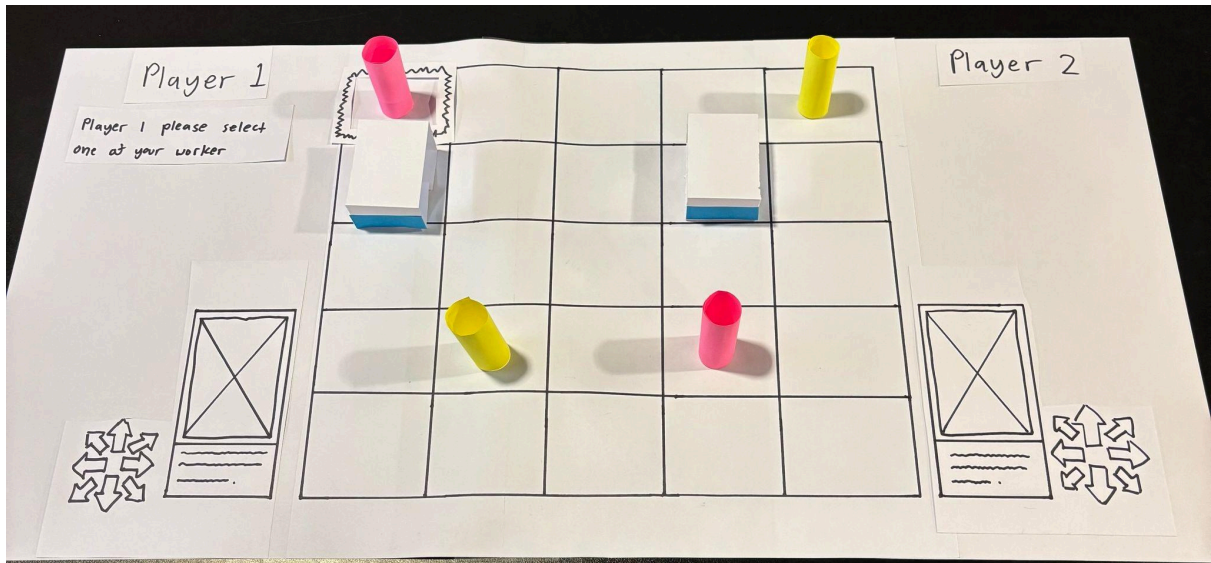


8.

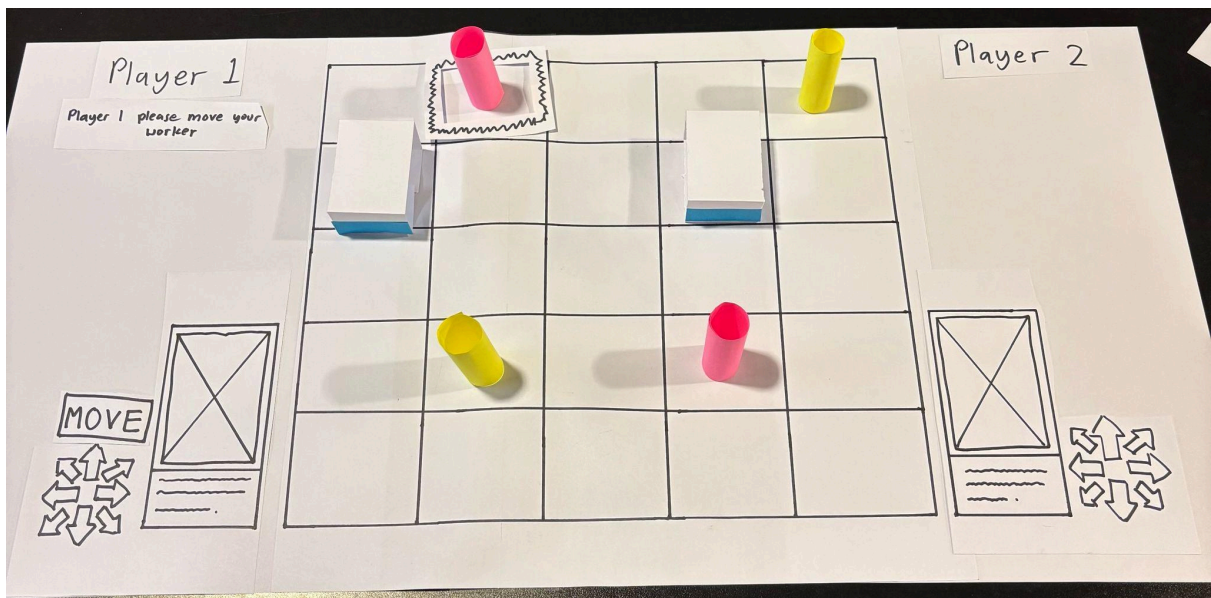


4.4. Step 9, 10, and 11 covers how player one can build on top of another building (second level)

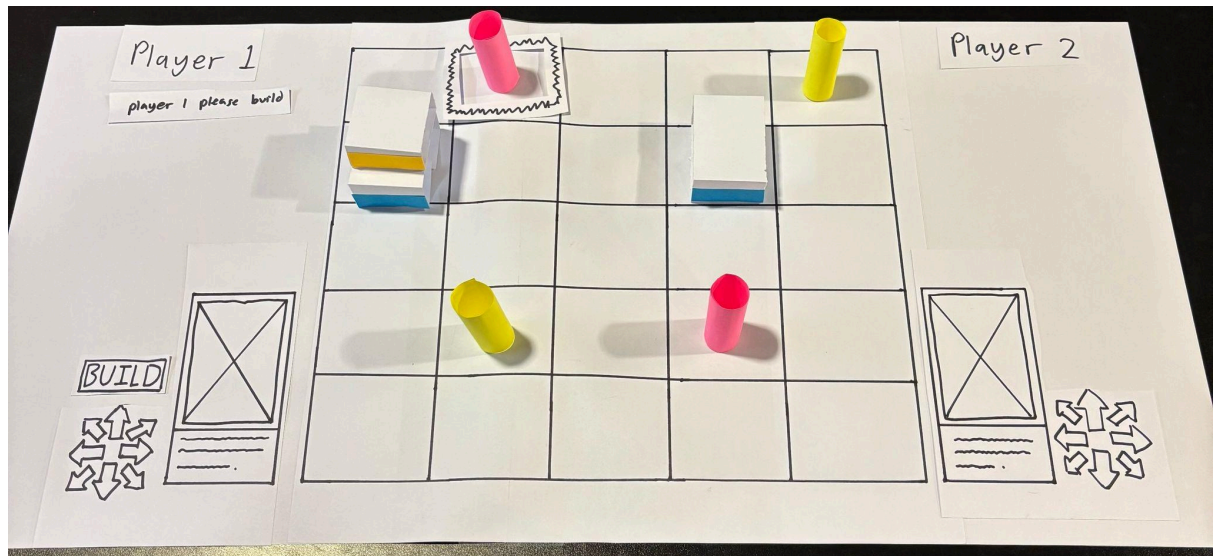
9.



10.

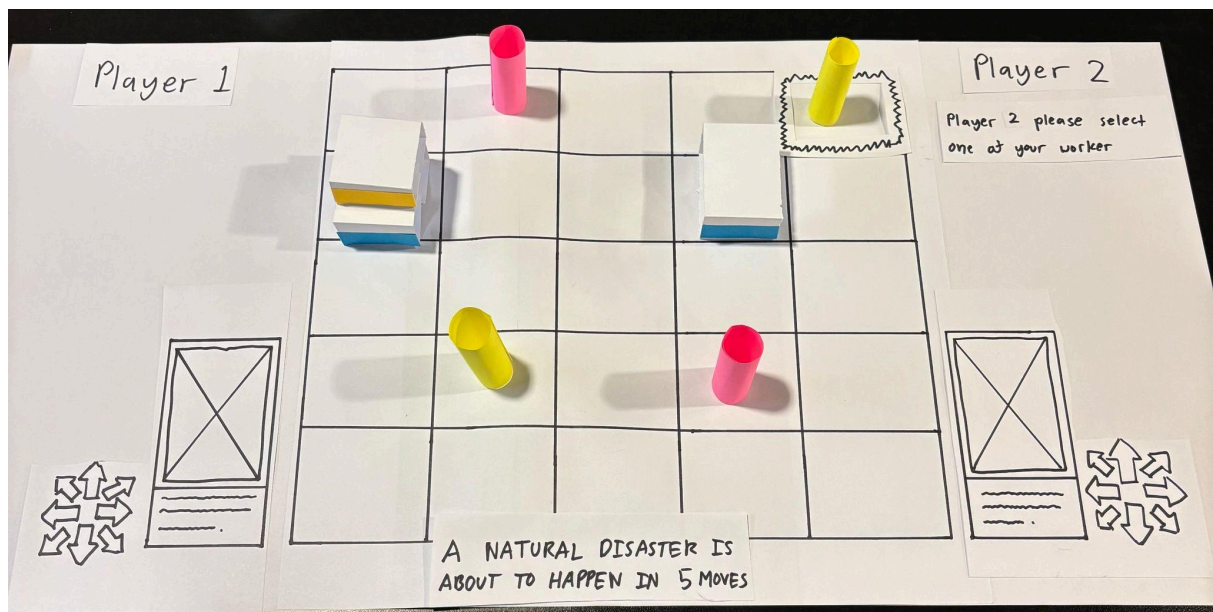


11.

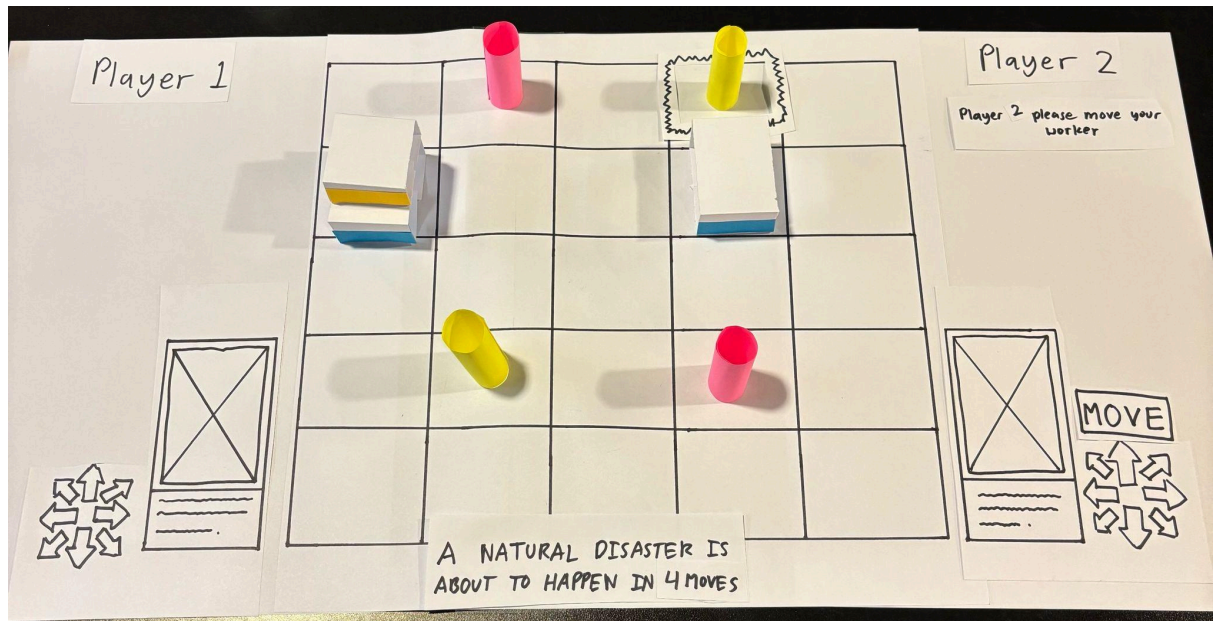


4.5. Step 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, and 25 shows how our chosen extension (natural disaster) would play out during a game

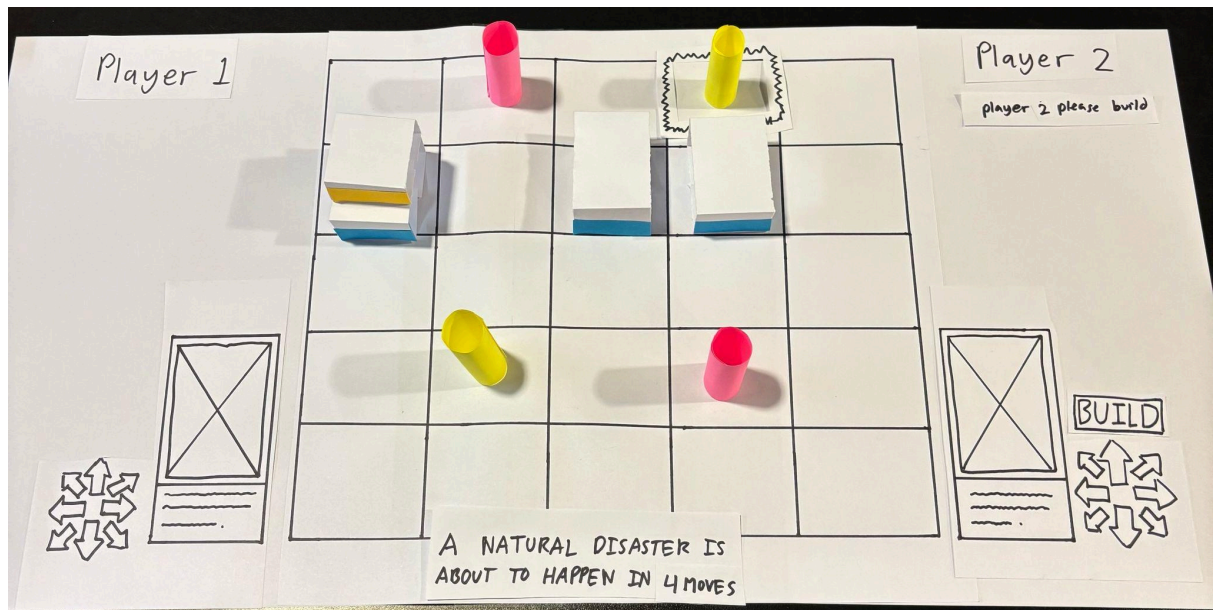
12.



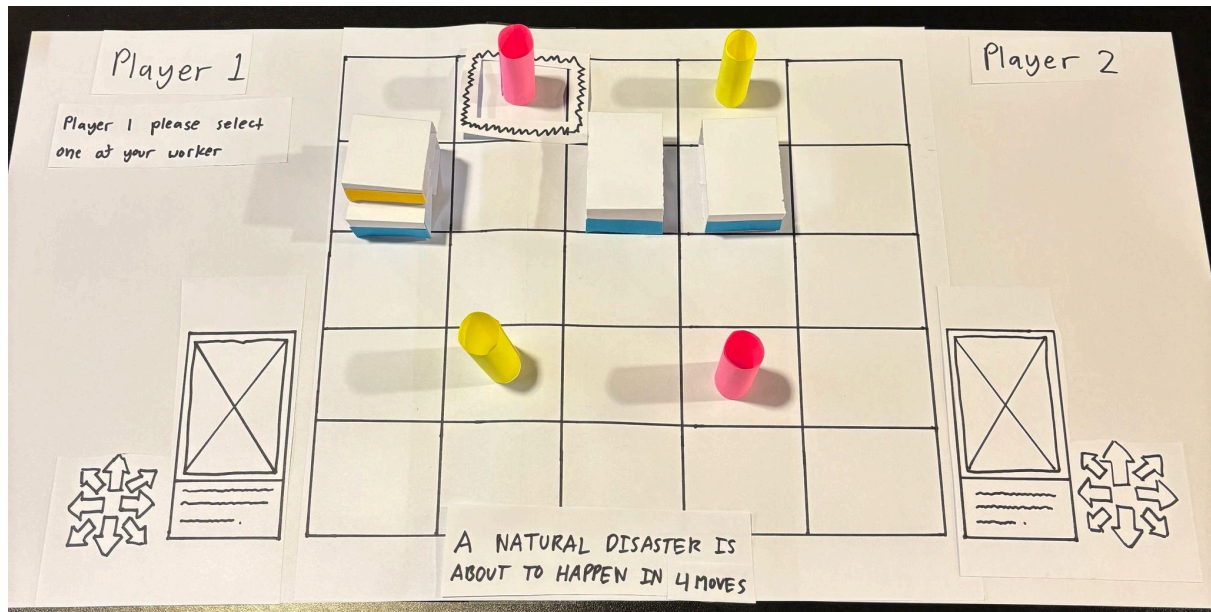
13.



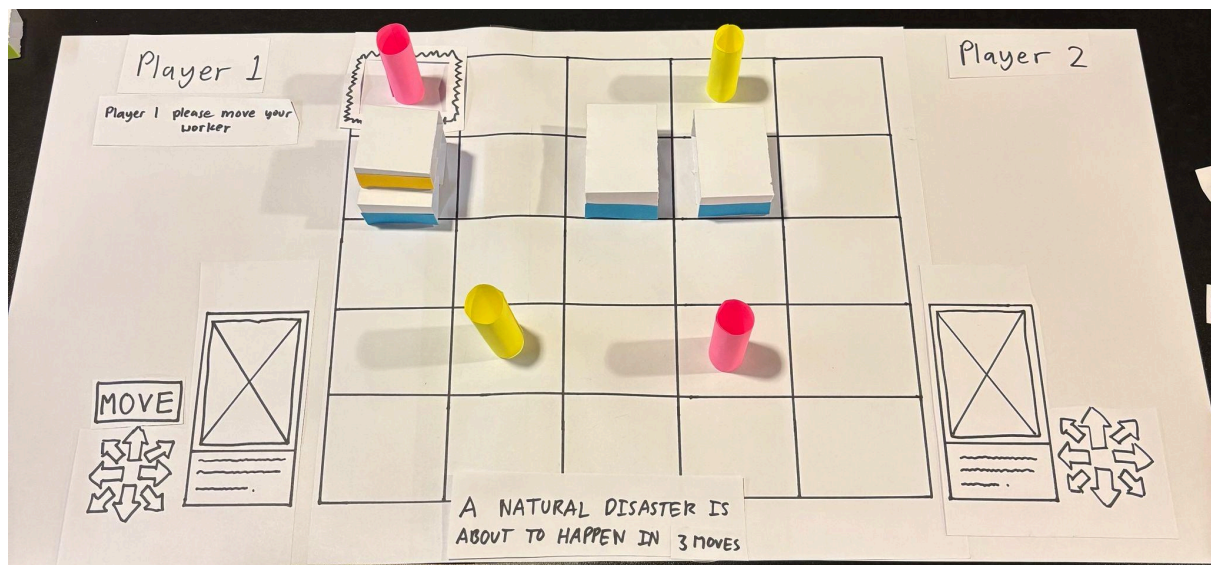
14.



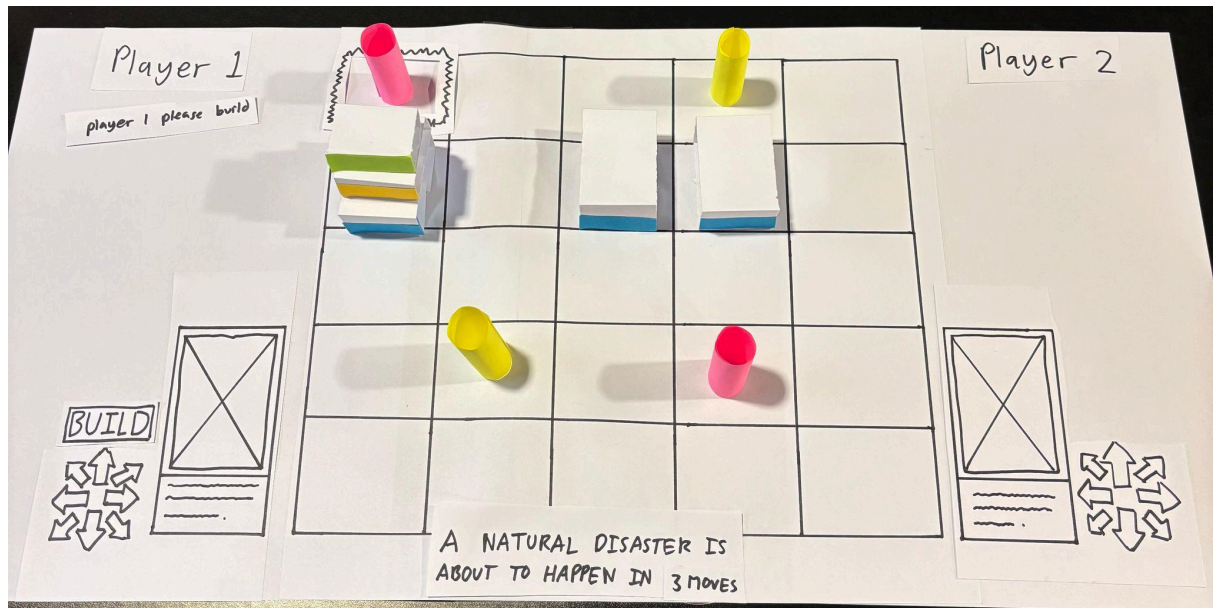
15.



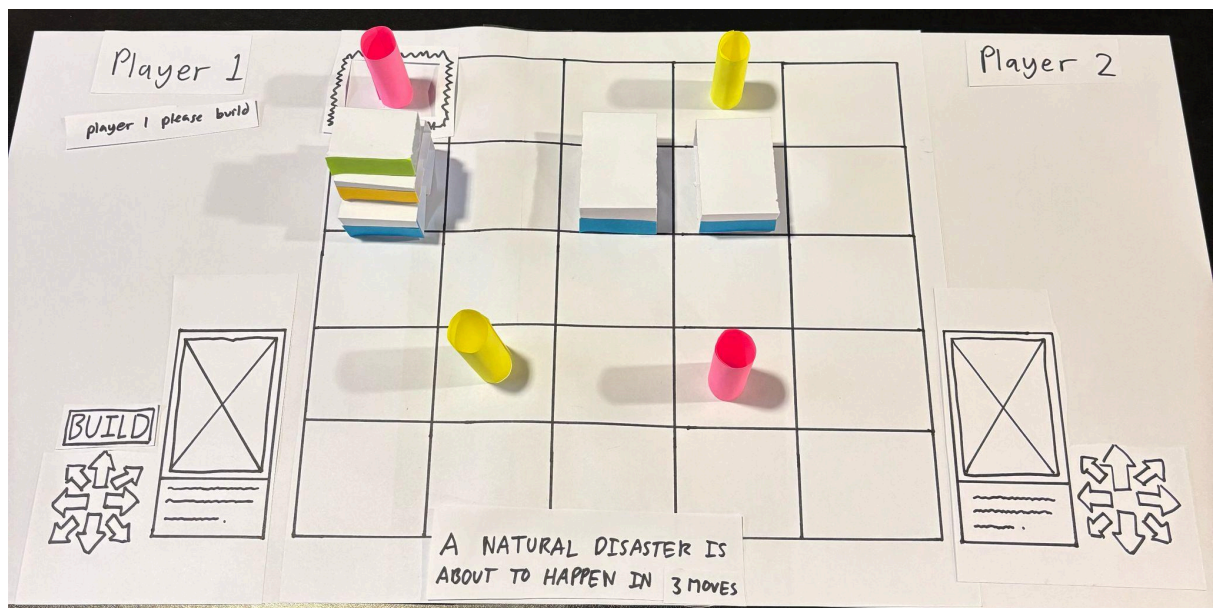
16.



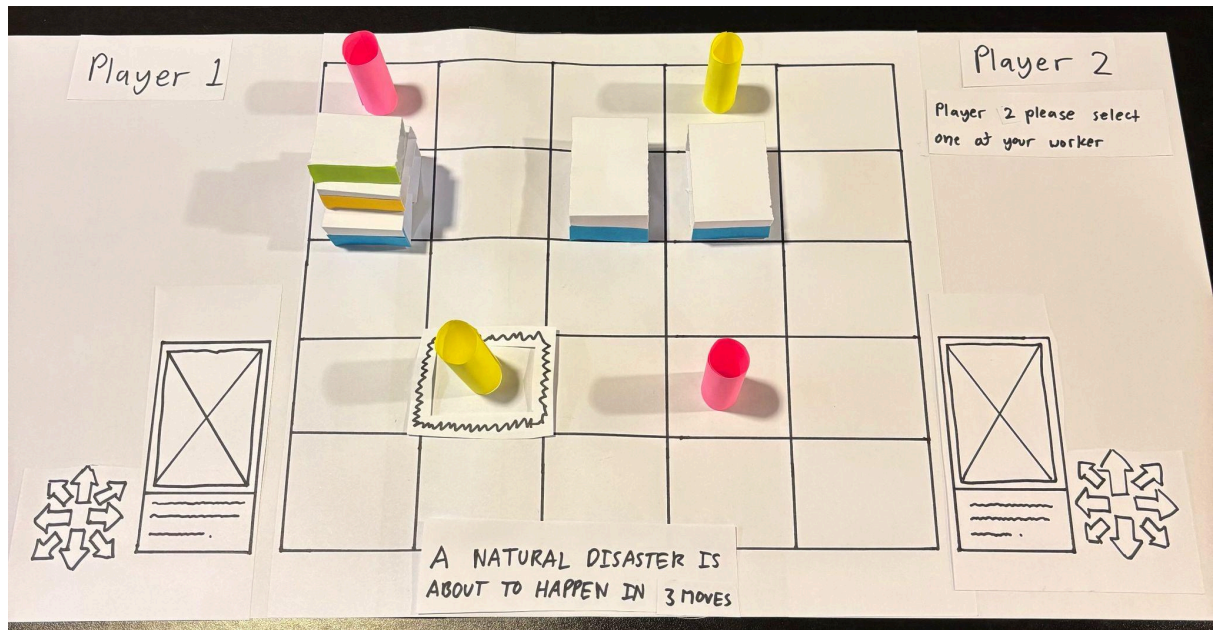
17.



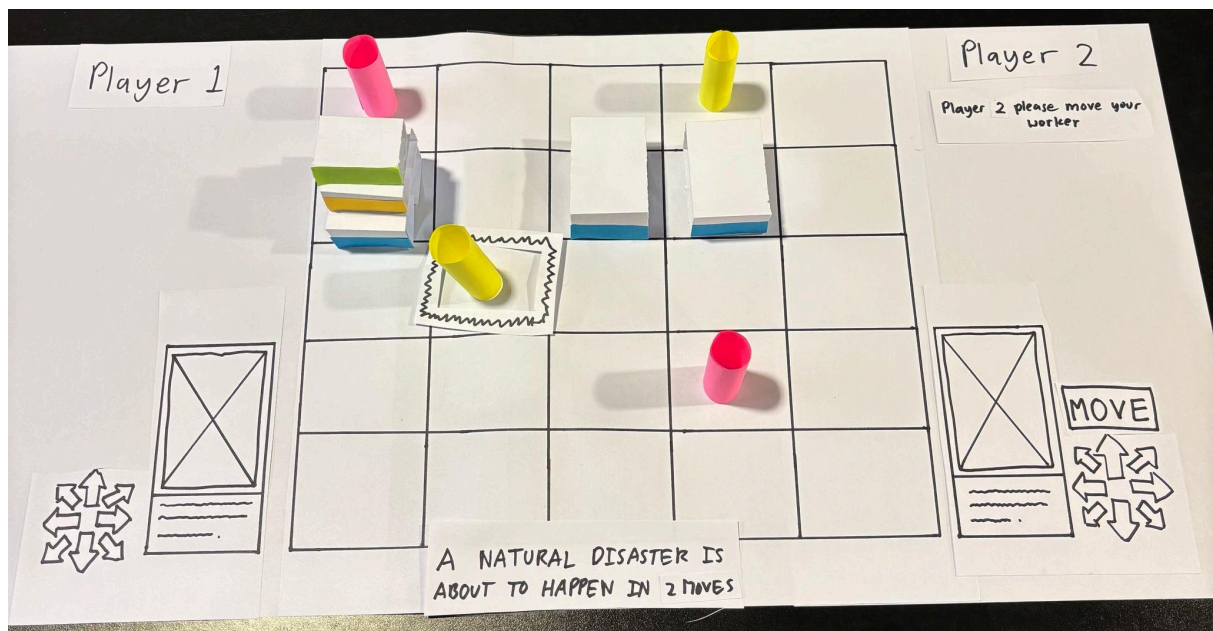
18.



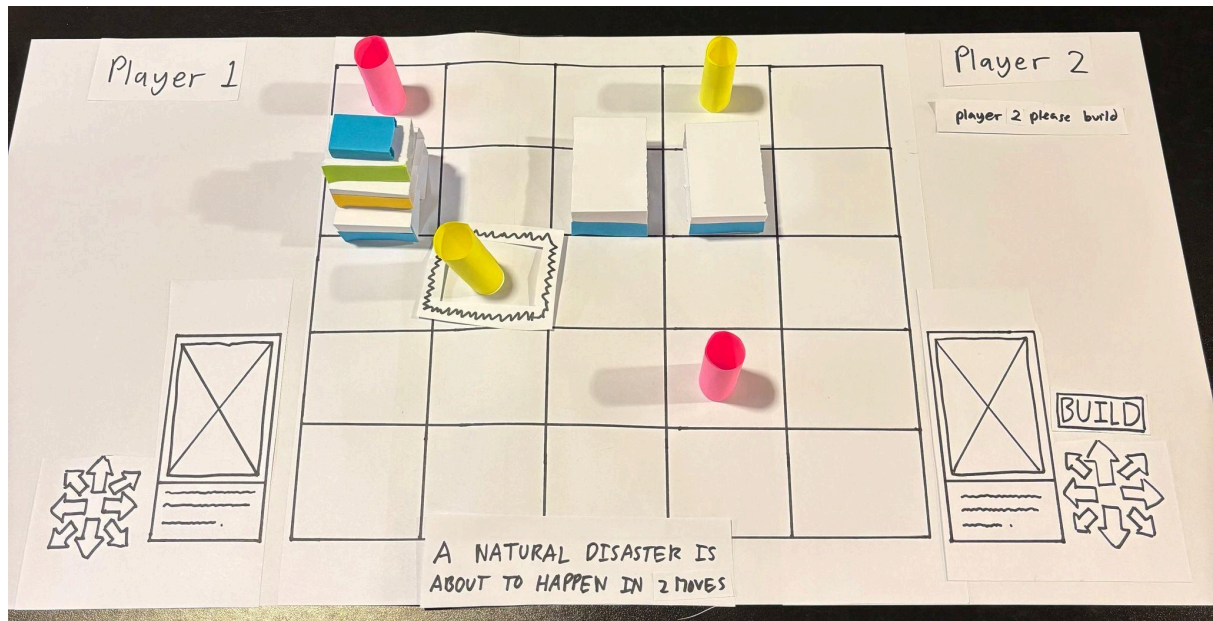
19.



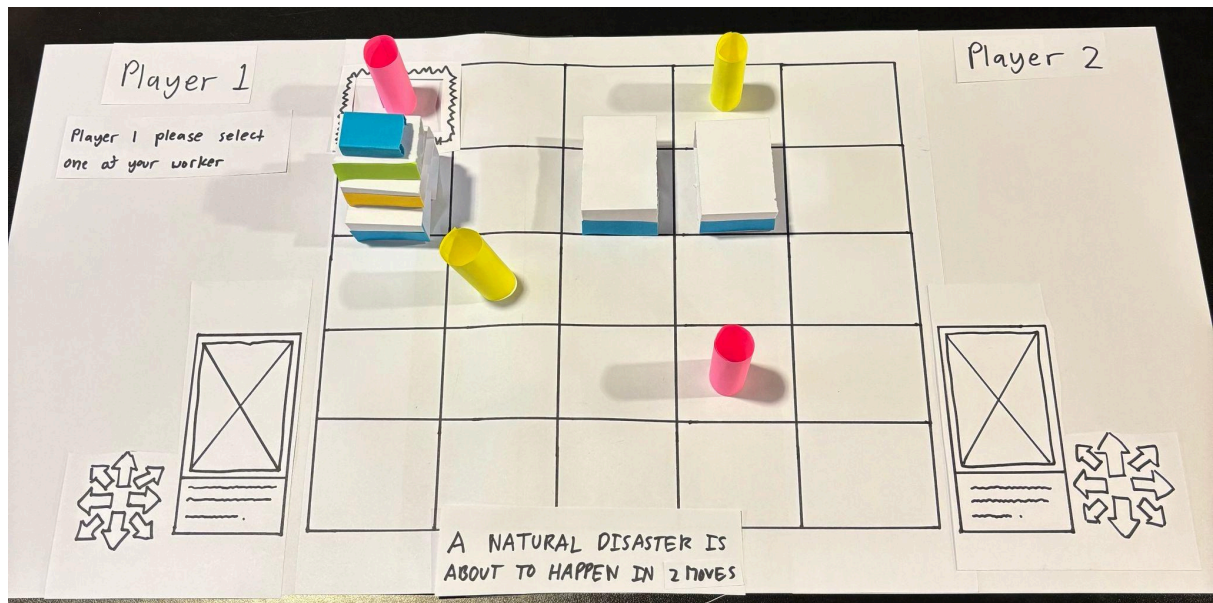
20.



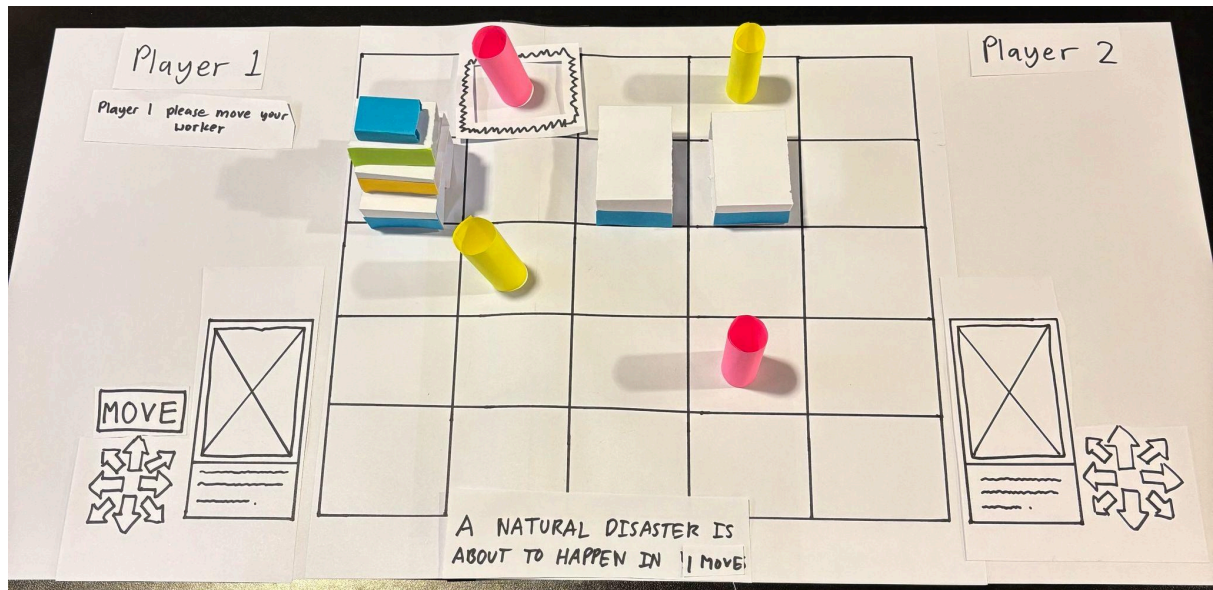
21.



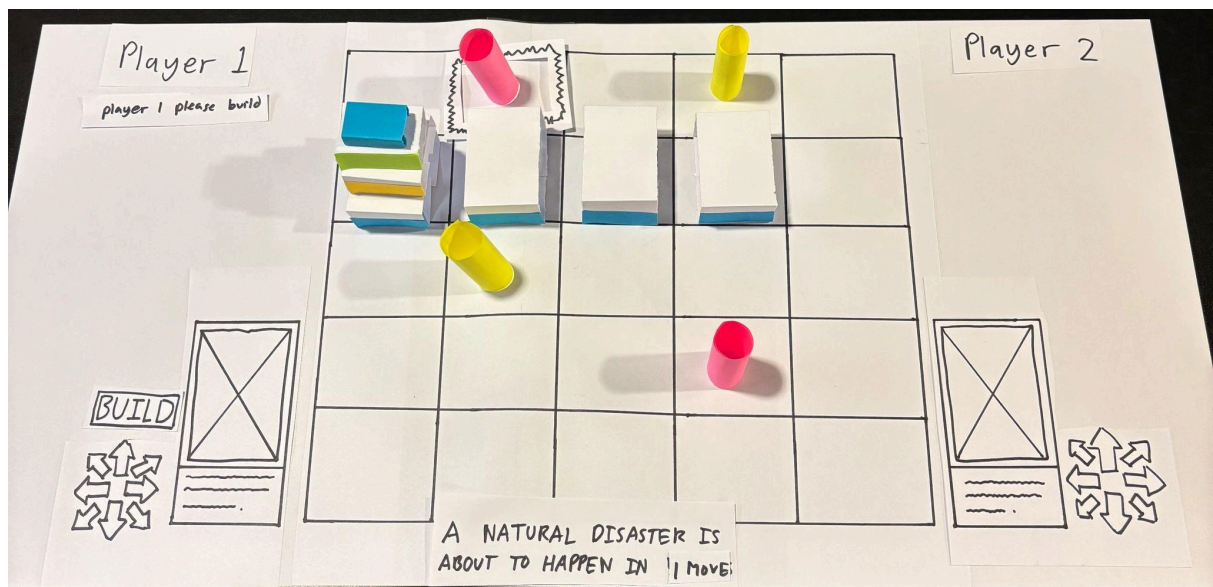
22.



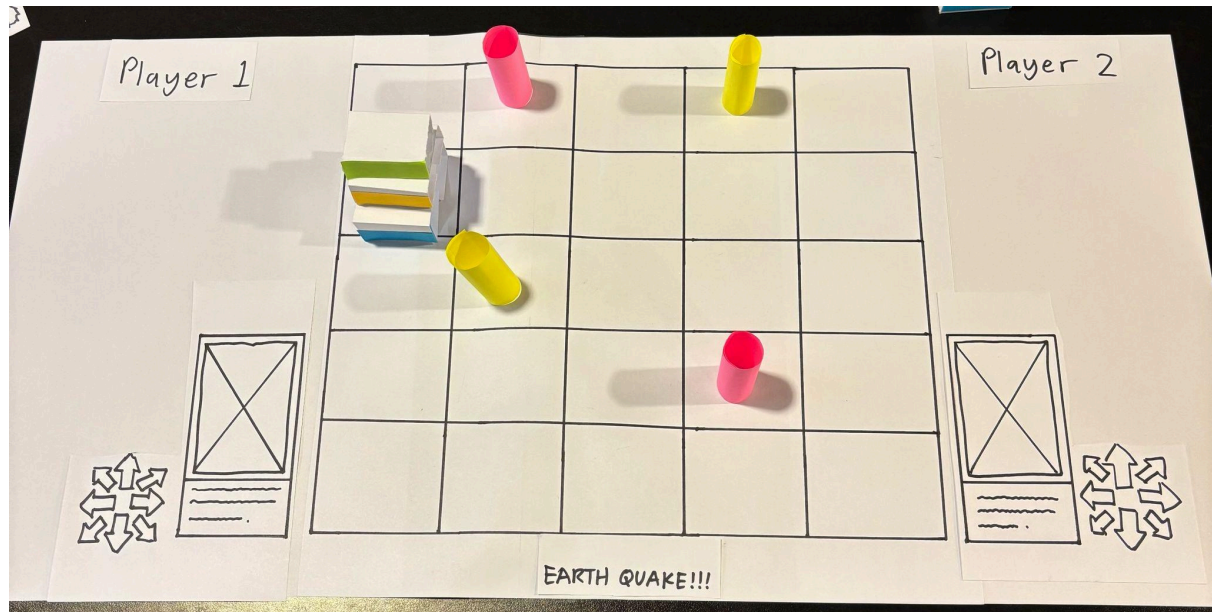
23.



24.

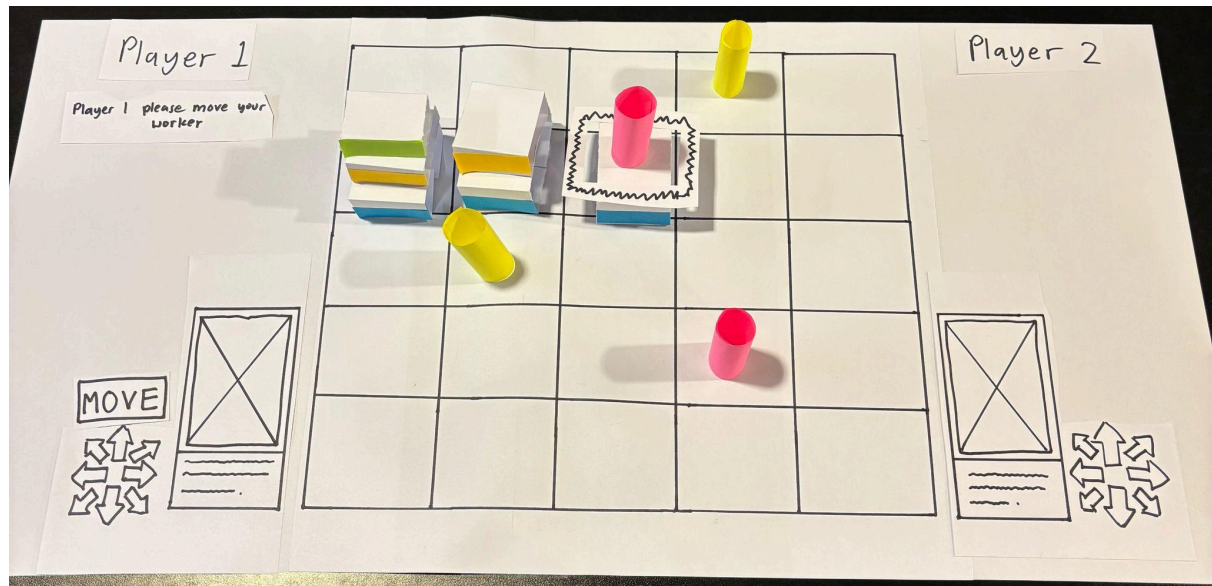


25.

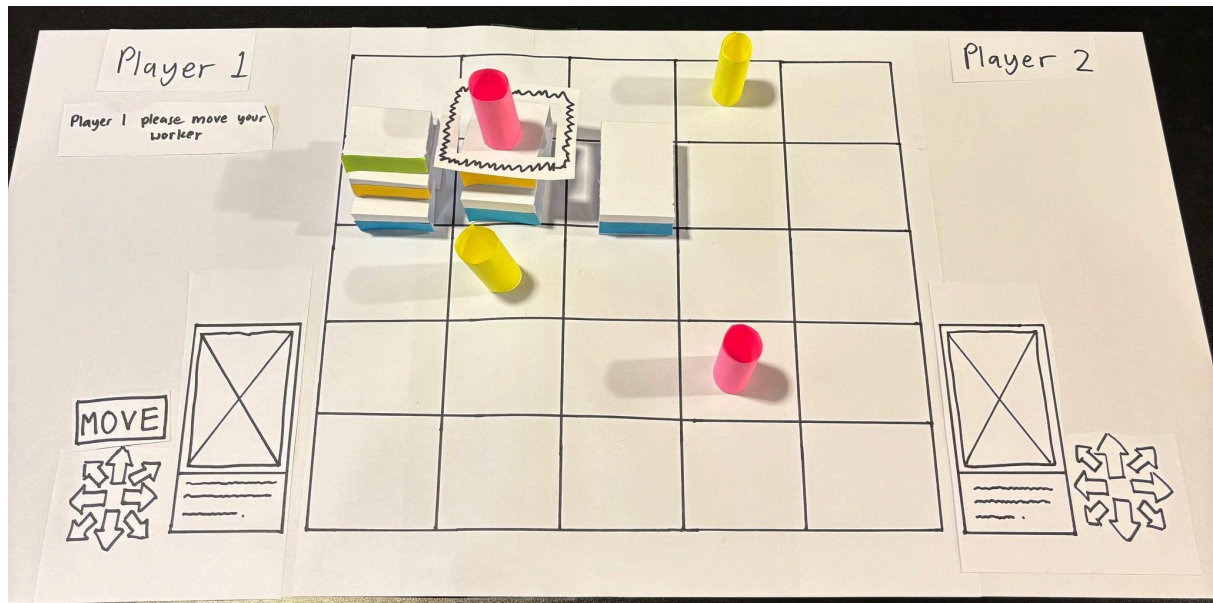


4.6. Step 26, 27, and 28 briefly shows how a player could win or lose in a game when the winner reaches the third building before the losing player

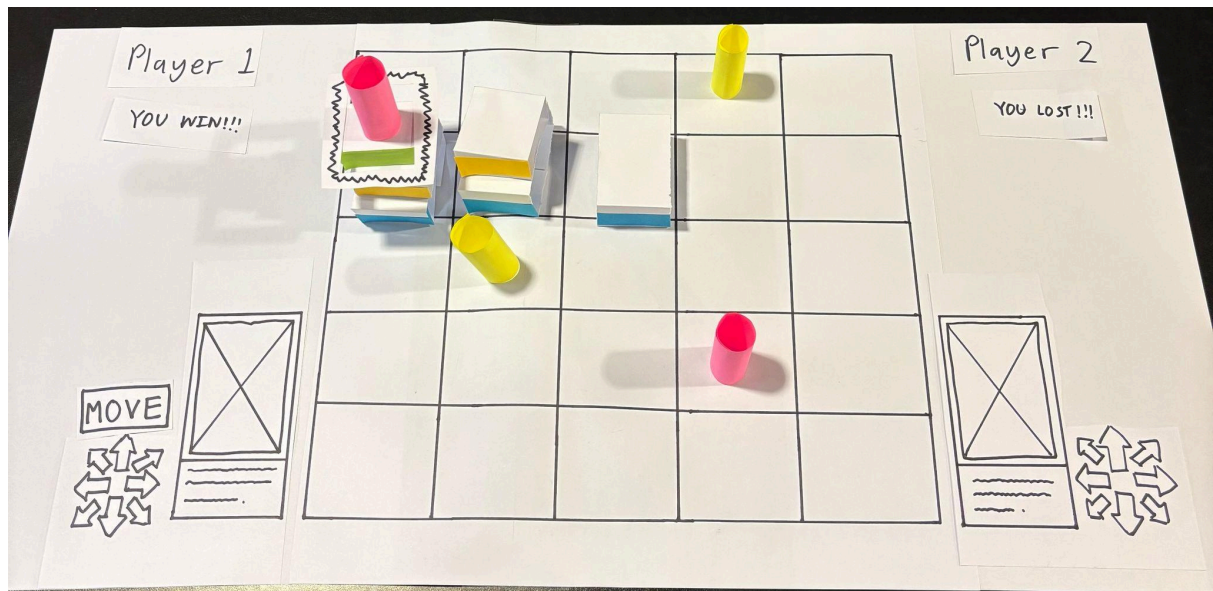
26.



27.

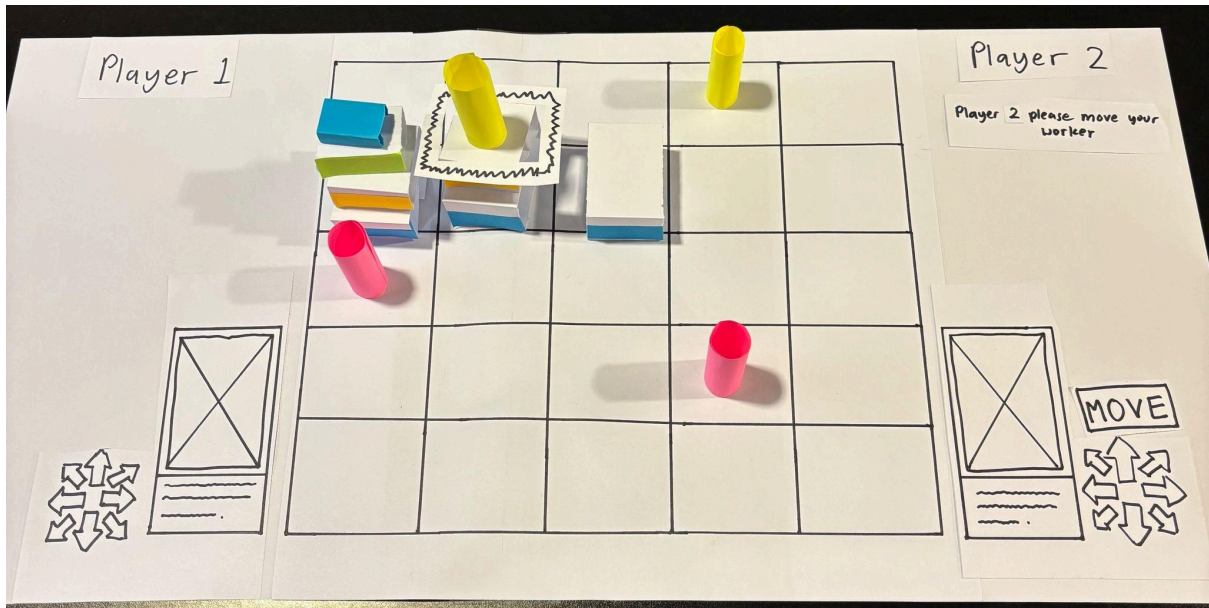


28.

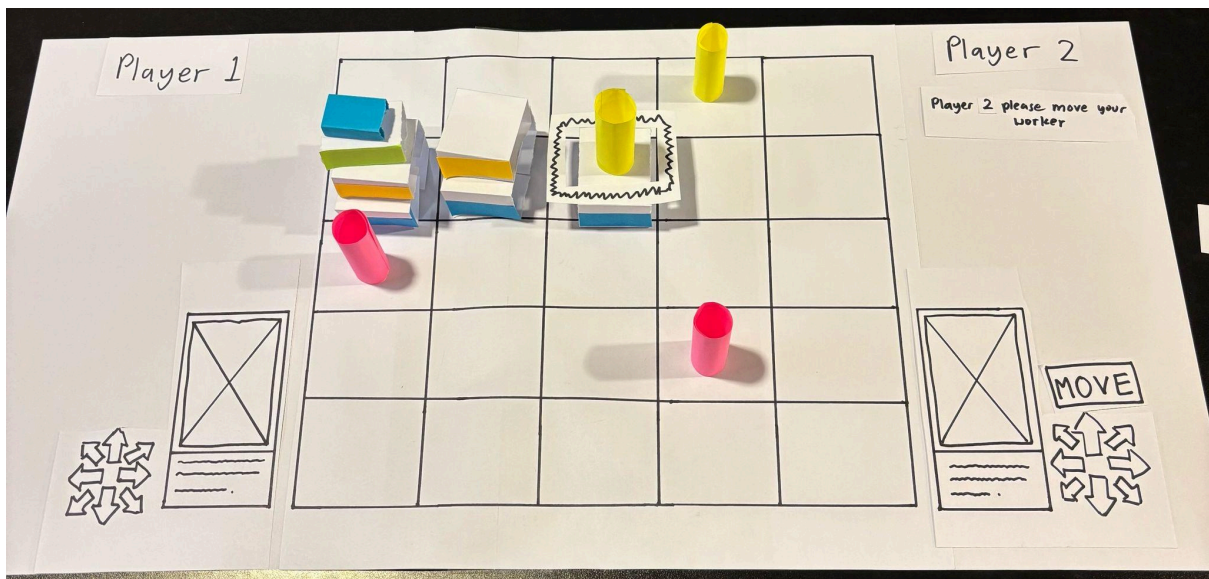


4.7. Step 29, 30, and 31 briefly shows how a player moves down from a building

29.



31.



32.

